SISTEMA OPERATIVO → INSIEME DI PROGRAMMI CHE FA DA INTERMEDIARIO TRA I PROGRAMMI E HARDWARE

LAVORA: IN MODALITÀ KERNEL (IN PARTE) E TUTTO IL RESTO GIRA IN MODALITÀ UTENTE

SCOPO: OFFRE UN MODELLO DEL CALCOLATORE PIÙ SEMPLICE E PULITO E GESTISCE LE RISORSE CONDIVISE TRA I PROGRAMMI.

2 FUNZIONI DEL SO

MACCHINA ESTESA (VISTA PROGRAMMATORE)

- OFFRE ASTRAZIONI PULITE → FILE, PROCESSI, ...
- OFFRE OPERAZIONI ASTRATTE PER I/O ED ESEGUIRE + PROGRAMMI.
- VISIONE TOP-DOWN (APPLICAZIONE → HARDWARE)

GESTORE/CONTROLLORE RISORSE (VISTA SISTEMA)

- ALLOCA IN MODO ORDINATO CPU, MEMORIA E DISPOSITIVI AI PROGRAMMI CHE LI CONTENDONO.
- MINIMIZZA I TEMPI MORTI.
- CONTROLLA GLI ACCESSI A DATI ALTRUI O DEL SO.

CONDIVIDERE LE RISORSE → MULTIPLEXING (CONDIVISIONE)

- TEMPO: LA RISORSA VIENE USATA A TURNO (STAMPANTE, CPU).
- SPAZIO: OGNI PROGRAMMA RICEVE UNA PARTE DELLA RISORSA (MEMORIA).

REGISTRI SPECIALI:

{ • PC • SP • PSW }

CONTEXT SWITCH → IL SO SALVA E RIPRISTINA QUESTI REGISTRI QUI.

DISPOSITIVI DI I/O: → DISPOSITIVO = CONTROLLER (ACCETTA COMANDI DAL OS) + HARDWARE

{ SO → DRIVER → CONTROLLER }

↳ 1 PER TIPO DI CONTROLLER E OS. (MODALITÀ KERNEL)

MENTRE UN PROCESSO ASPETTA IL DISCO, LA CPU NE ESEGUE UN ALTRO

---

2 MODALITÀ DELLA CPU

INDICATI DAI BIT IN PSW

UTENTE

- SOTTOINSIEME DI ISTRUZIONI: ARITM, LOGICHE, SALTI, ACCESSO SOLO ALLA PROPRIA MEMORIA.

KERNEL

- TUTTE LE ISTRUZIONI, PRIVILEGIATE, GESTIONE DI INTERRUPT E MMU.

ISTRUZIONI PRIVILEGIATE →

```
esegui(istr):
  if privilegiata(istr) and PSW.modo == UTENTE:
    trap(ISTR_PRIVILEGIATA)    // la CPU genera un'eccezione, interviene il SO
  else:
    // ... esecuzione normale
```

- ISTRUZIONI I/O A BASSO LIVELLO
- IMPOSTARE I REGISTRI DELLA MMU
- DISABILITARE GLI INTERRUPT

} ISTRUZIONI PRIVILEGIATE

COME SI ENTRA NEL KERNEL? (3 CASI):

- TRAP PER ERRORE → IL PROCESSO TENTA UN' AZIONE NON PERMESSA
- CHIAMATA DI SISTEMA → IL PROCESSO SI FA "INTRAPPOLARE" PER RICHIEDERE UN SERVIZIO
- INTERRUPT HARDWARE → UN DISPOSITIVO SEGNALA UN EVENTO

IL SO INTERVIENE QUANDO LO ATTIVA UN EVENTO:

- INTERRUPT (HARDWARE): ASINCRONO, UN DISPOSITIVO HA TERMINATO O RICHIEDE ATTENZIONE.
- TRAP (SOFTWARE): SINCRONO, ERRORE DEL PROGRAMMA O CHIAMATA DI SISTEMA.

GESTIONE INTERRUPT:

1) IL DRIVER SCRIVE SUI REGISTRI DEL CONTROLLER
2) IL CONTROLLER INVIA UN SEGNALE DI INTERRUPT AL GESTORE DEGLI INTERRUPT.
3) SE LIBERO, LO INOLTRA ALLA CPU E COMUNICA L'ID DEL DISPOSITIVO.
4) LA CPU TERMINA L'ISTRUZIONE CORRENTE, SALVA LO STATO E SALTA AL GESTORE/HANDLER INDICATO DAL VETTORE DEGLI INTERRUPT.
5) TERMINA LA GESTIONE, RIPRENDE IL PROGRAMMA CORRENTE.

COSA FA LA CPU QUANDO RICEVE UN INTERRUPT?

1) COMPLETA L'ISTRUZIONE MACCHINA IN CORSO.
2) SALVA IN MEMORIA LO STATO.
3) SALTA AL GESTORE DI QUEL INTERRUPT.
4) DOPO LA GESTIONE, RIPRISTINA LO STATO: IL PROGRAMMA PROSEGUE COME NULLA.

L'INTERRUPT È TRASPARENTE.

```
while(!halt){
  if(interrupt && !int_disable){
    salva stato;
    invia INTA;
    leggi_device_id;
    int_disable = true;  // disabilita altri interrupt
    modo = KERNEL;
    PC = int_vector[devide_id]  // tabella indirizzi dei gestori
  }
  fetch istruzione;
  decode istruzione;
  esegui istruzione;
}
```

DISINT

DISABILITA GLI INTERRUPT ALMENO QUESTA PORZIONE DI CODICE NON VIENE ESEGUITA (NON SOVRASCRIVE LO STATO E PERMETTE DI FARE UN INTERRUPT ALLA VOLTA)

ALLA FINE ESEGUE RETINT CHE RIABILITA GLI INTERRUPT, RIPRISTINA PSW E LO STATO.

↳ PER RITORNARE IN MODALITÀ UTENTE.

CHIAMATE DI SISTEMA → SONO IL MECCANISMO CON CUI I PROGRAMMI RICHIEDONO SERVIZI AL SO, OVVERO ISTRUZ. AGGIUNT. DEL LIVELLO 3 (POSIX → 100 ISTRUZ.)

↓

SEMPLIFICANO L'USO DELLE RISORSE, E COORDINARE PROGRAMMI ESEGUITI CONTEMPORANEAMENTE.

COME FUNZIONANO?

1) IL PROGRAMMA CHIAMA UNA FUNZIONE C → UN WRAPPER FORNITO DALLO STANDARD POSIX.
2) QUELLA FUNZIONE ESEGUE UNA TRAP → PASSARE IN MODALITÀ KERNEL.
3) IL KERNEL OFFRE IL SERVIZIO → SI RITORNA AL PROGRAMMA.

- **CHIAMATA DI SISTEMA** → ENTRA NEL KERNEL, POTENTE MA COSTA (CAMBIO MODALITÀ).
- **FUNZIONE DI LIBRERIA** (PRINTF) → RESTA IN MODALITÀ UTENTE, AL SUO INTERNO USA UNA WRITE (CHIAMATA DI SISTEMA), MA PRIMA BUFFERIZZA: ACCUMULA I DATI E POI LA MANDA A WRITE 1 VOLTA SOLA.

```
n = read(fd, buffer, nbytes)
```

- n → NUMERO BYTE LETTI (≥0, -1)
- fd → FILE
- buffer → DOVE METTERE I DATI
- nbytes → QUANTI BYTES AL MASSIMO.

PROCESSO → UN PROGRAMMA IN ESECUZIONE, CONTIENE TUTTO CIÒ CHE SERVE PER ESEGUIRLO, COME CODICE, REGISTRI (PC, SP), FILE APERTI.

- PUÒ CREARE NUOVI PROCESSI → ALBERO DEI PROCESSI.
- OGNI PROCESSO HA LA PROPRIA MEMORIA PRIVATA: NESSUNO PUÒ LEGGERE O MODIFICARE I DATI AL SUO INTERNO, È DIVISA IN 4 ZONE:

↓

MEMORIA PRIVATA:

- **STACK** → VARIABILI LOCALI, CHIAMATE DI FUNZIONE.
- **HEAP** → MEMORIA DINAMICA RICHIESTA A RUNTIME.
- **DATI** → VARIABILI GLOBALI E COSTANTI
- **CODICE** → ISTRUZ. DEL PROGRAMMA

FILE E DIRECTORY → IL SO OFFRE IL FILE PER SALVARE IN MODO UNIFORME I DATI.

- LE DIRECTORY CONTENGONO FILE E ALTRE DIRECTORY (FILE SYSTEM).
- PIPE → UN FILE SPECIALE PER FAR COMUNICARE 2 PROCESSI.

PROTEZIONI → OGNI FILE HA PERMESSI SU 3 LIVELLI: PROPRIETARIO | GRUPPO | ALTRI

OGNUNO HA 3 AZIONI → R (READ) | W (WRITE) | X (EXECUTE)

SHELL → PROGRAMMA CHE LEGGE I TUOI COMANDI E LI ESEGUE, NON FA PARTE DEL SO, USA SOLO LE CHIAMATE DI SISTEMA (FORK, EXEC, WAIT).

CI SONO 4 ARCHITETTURE DEL SO:

- MONOLITICO → TUTTO IL SO È UN UNICO BLOCCO IN MOD. KERNEL.
- A STRATI → LIVELLI ORDINATI, OGNUNO USA QUELLO SOTTO.
- MICROKERNEL → NEL KERNEL SOLO L'ESSENZ. DRIVER E FILE SYSTEM SONO PROCESSI UTENTE.
- MACCHINA VIRTUALE → UN SO COMPLETO GIRA SOPRA UN HYPERVISOR.

CHIAMATE DI SISTEMA IN C →

- LIBRERIA: <UNISTD.H>
- STRACE ./PROG → MOSTRA TUTTE LE CHIAMATE DI SISTEMA
- STRACE -C → LE CONTA.

PRINTF VS WRITE →

PRINTF: SU TERMINALE L'OUTPUT LO VEDI SUBITO (SVUOTA IL BUFFER AD OGNI \N) MA SE DIREZIONO L'OUTPUT SU UN FILE IL BUFFER SI SVUOTA SOLO QUANDO È PIENO O IL PROGRAMMA È TERMINATO.

↓

QUINDI SCRIVERE UN CARATTERE ALLA VOLTA CON WRITE È COSTOSO, DATO CHE FA UNA CHIAMATA DI SISTEMA OGNI VOLTA, INVECE PRINTF O PUTCHAR FANNO MENO CHIAMATE.

WRITE: SCRIVE SEMPRE SUBITO.

GESTIONE DEGLI ERRORI:

PER CONVENZIONE UNIX SE FALLISCE UNA CHIAMATA VIENE RESTITUITO -1 E SCRIVE DENTRO ERRNO, PERROR("") NE LEGGE IL CONTENUTO:

```c
int check(int res, const char *msg){
  if(res == -1){
    perror(msg);   // stampa l'errore con una etichetta iniziale
    exit(1);       // esce con stato 1
    return res;
  }
}

int fd = check(open("file.txt", O_RDONLY), "open");
```

CON QUESTA FUNZIONE CHECK CONTROLLO SEMPRE IL VALORE DI RITORNO.

FILE E DESCRITTORI:

UN FILE È RAPPRESENTATO DA UN NUMERO: UN DESCRITTORE.

0	STANDARD INPUT
1	STANDARD OUTPUT
2	STANDARD ERROR
3	NUOVO DESC. DI OPEN

FLAG	SIGNIFICATO
O_RDONLY / O_WRONLY / O_RDWR	→ LEGGI / SCRIVI / ENTRAMBE
O_CREAT	→ CREA IL FILE SE NON ESISTE
O_TRUNC	→ SVUOTA IL FILE SE ESISTE GIÀ
O_APPEND	→ SCRIVI SEMPRE IN FONDO

PUOI CONCATENARE I FLAG CON L'OR ('|'):

```c
int fd = check(open("file.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644), "open");
```

CON O_CREAT SERVE LA MODE, OVVERO I PERMESSI DEL NUOVO FILE.

```c
n = read(fd, buf, count)   // legge dal file nel buffer
n = write(fd, buf, count)  // scrive dal buffer nel file
```